SISTEMAS EMBEBIDOS

Universidad Politécnica del

Bicentenario.

Reporte Practica 2. Control ON-OFF de nivel.

Ingeniería Mecatronica 701

Edgar Uriel Rodriguez Moreno 24033660

Jorge Zamorano Chavez 24033247

Axel Gabriel Domínguez Macias 24032952

Braulio Vázquez Morado 24032975

Daisy Jackeline Hernandez Ramos 24033069

Silao, Guanajuato a 20 de Octubre de 2026.

Introducción

El control automatizado de fluidos es un pilar fundamental en la ingeniería aplicada y en los procesos industriales y domésticos, permitiendo la gestión eficiente de recursos hídricos en cisternas y tanques elevados. El presente reporte describe el diseño, la programación y la validación experimental de un prototipo de automatización de nivel de agua basado en el microcontrolador ESP32.

El sistema implementa una interfaz de usuario mediante tres pulsadores para configurar de manera dinámica los umbrales de nivel máximo y nivel mínimo, un sensor ultrasónico HC-SR04 para la medición continua de la distancia al espejo de agua, la simulación lógica de bombas de llenado y vaciado, y una pantalla LCD para la visualización en tiempo real de los porcentajes actuales y configurados. Cabe destacar que el desarrollo del firmware se realizó sin hacer uso del ecosistema o librerías de Arduino, demostrando el control directo de los periféricos del microcontrolador.

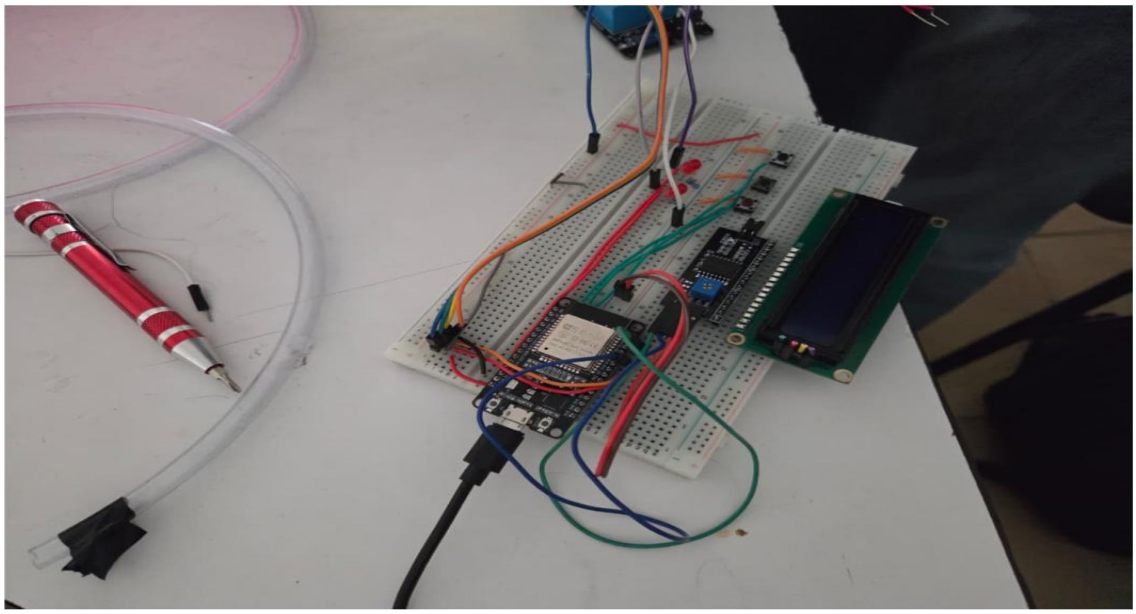
Marco Teórico

Adentrándonos en los fundamentos del marco teórico que sustentan este proyecto es indispensable analizar cada uno de los componentes de hardware involucrados comenzando por el microcontrolador ESP32 el cual consiste en un sistema en un chip de bajo costo y bajo consumo de energía con conectividad inalámbrica integrada desarrollado por Espressif Systems basado en un procesador de 32 bits que actúa como la unidad central de procesamiento encargada de leer las señales

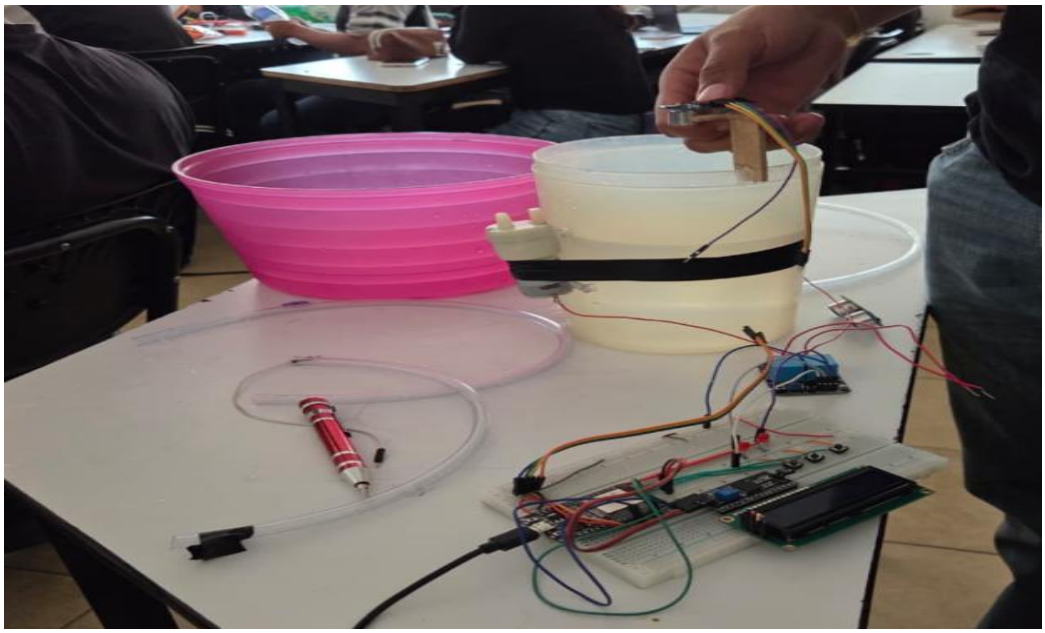
de los botones y del sensor ultrasónico procesar la lógica de control y enviar los datos a la pantalla operando mediante pines de entrada y salida configurados directamente en C y C++ utilizando el framework nativo ESP-IDF sin requerir librerías externas de Arduino. Otro elemento clave es el sensor de proximidad ultrasónico HCSR04 un módulo que utiliza ondas acústicas a una frecuencia específica para medir distancias sin contacto físico aplicando el principio del tiempo de vuelo del eco para calcular la distancia entre la parte superior del tanque y la superficie del agua permitiendo que el firmware determine el porcentaje de llenado actual mediante disparos de pulso y medición de eco compatibles con lenguajes como C y C++. Asimismo la pantalla LCD basada en tecnología de cristal líquido y acoplada a un adaptador I2C sirve para presentar de forma visual e intuitiva al operador el porcentaje actual de agua y los valores umbral configurados conectándose a los pines de comunicación dedicados del microcontrolador. Complementando la interfaz se encuentran los Pulsadores dispositivos electromecánicos momentáneos que permiten la interacción manual de tal manera que el primer botón conmuta la selección entre la variable máxima hl y la mínima ll mientras que el segundo y tercer botón incrementan o decrementan el valor porcentual seleccionado conectándose a pines con resistencia de pullup interno.

Desarrollo de la practica

Durante el desarrollo y la implementación del sistema físico se interconectaron todos estos elementos configurando el pin de disparo y eco del sensor ultrasónico los tres botones con pullup interno y la pantalla LCD por el bus I2C además de establecer las salidas digitales para simular la activación de las bombas.



La lógica de control opera de manera continua de modo que al pulsar el primer botón el usuario alterna entre la edición del nivel máximo y mínimo ajustando los valores porcentuales con los otros dos botones. El sensor ultrasónico mide la distancia que se mapea a un porcentaje y cuando el nivel actual desciende al valor mínimo establecido se activa la bomba de llenado mientras que al alcanzar el nivel máximo se desactiva el llenado y se activa la bomba de vaciado logrando un ciclo automatizado preciso. El código implementado directamente en Arduino estructurado para ESP32 dev module gestiona las tareas de lectura y control sin intermediarios de librerías de Arduino.



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ==========================================
// DEFINICIÓN DE PINES (ESP32 de 30 pines)
// ==========================================

#define TRIG_PIN 5
#define ECHO_PIN 18   // en físico, con divisor de voltaje 1k + 2k

#define SDA_PIN 21
#define SCL_PIN 22

#define LED1 26       // LED 1: subiendo agua (llenando)
#define LED2 27       // LED 2: nivel alto

#define BTN1 32
#define BTN2 33
#define BTN3 25

// ==========================================
// CALIBRACIÓN DEL TANQUE (en cm)
// ==========================================

#define DIST_LLENO 7.0    // 7 cm  -> 100%
#define DIST_VACIO 19.0   // 19 cm -> 0%

// Umbrales de los LEDs (en porcentaje)
#define PORC_BAJO 20      // por debajo: se enciende LED1 (subiendo agua)
#define PORC_ALTO 90      // por encima: se enciende LED2 (nivel alto)

// ==========================================
// CONFIGURACIÓN DEL LCD
```

```
// ========================================

#define LCD_ADDRESS 0x27
#define LCD_COLUMNS 16
#define LCD_ROWS 2

LiquidCrystal_I2C lcd(LCD_ADDRESS, LCD_COLUMNS, LCD_ROWS);

// ========================================
// VARIABLES
// ========================================

long tiempo;
float distancia;
float porcentaje;

// ========================================
// CONFIGURACIÓN INICIAL
// ========================================

void setup() {

  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(BTN1, INPUT_PULLUP);
  pinMode(BTN2, INPUT_PULLUP);
  pinMode(BTN3, INPUT_PULLUP);

  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, LOW);

  Wire.begin(SDA_PIN, SCL_PIN);

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("Control de nivel");
  lcd.setCursor(0, 1);
  lcd.print("Iniciando...");

  delay(2000);
  lcd.clear();
}

// ========================================
// PROGRAMA PRINCIPAL
// ========================================

void loop() {

  // Pulso de disparo
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
```

```
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Medición
  tiempo = pulseIn(ECHO_PIN, HIGH, 30000);
  distancia = (tiempo * 0.0343) / 2.0;

  // ----------------------------------------
  // Conversión a porcentaje (7 cm = 100%, 19 cm = 0%)
  // ----------------------------------------

  porcentaje = (DIST_VACIO - distancia) * 100.0 / (DIST_VACIO -
DIST_LLENO);

  if (porcentaje > 100.0) porcentaje = 100.0;
  if (porcentaje < 0.0)   porcentaje = 0.0;

  // ----------------------------------------
  // Lógica de LEDs (solo si la lectura es válida)
  // ----------------------------------------

  if (tiempo != 0) {

    if (porcentaje <= PORC_BAJO) {
      // Nivel bajo: se está llenando
      digitalWrite(LED1, HIGH);
      digitalWrite(LED2, LOW);
    }
    else if (porcentaje >= PORC_ALTO) {
      // Nivel alto
      digitalWrite(LED1, LOW);
      digitalWrite(LED2, HIGH);
    }
    // Entre ambos umbrales: se mantiene el estado anterior
  }

  // ----------------------------------------
  // LCD
  // ----------------------------------------

  lcd.setCursor(0, 0);
  lcd.print("Nivel:          ");
  lcd.setCursor(7, 0);

  if (tiempo == 0) {
    lcd.print("---");
  } else {
    lcd.print((int)porcentaje);
    lcd.print("%");
  }

  lcd.setCursor(0, 1);
  if (digitalRead(LED1)) {
    lcd.print("Subiendo agua   ");
  } else if (digitalRead(LED2)) {
    lcd.print("Nivel alto      ");
  } else {
    lcd.print("Esperando...    ");
```

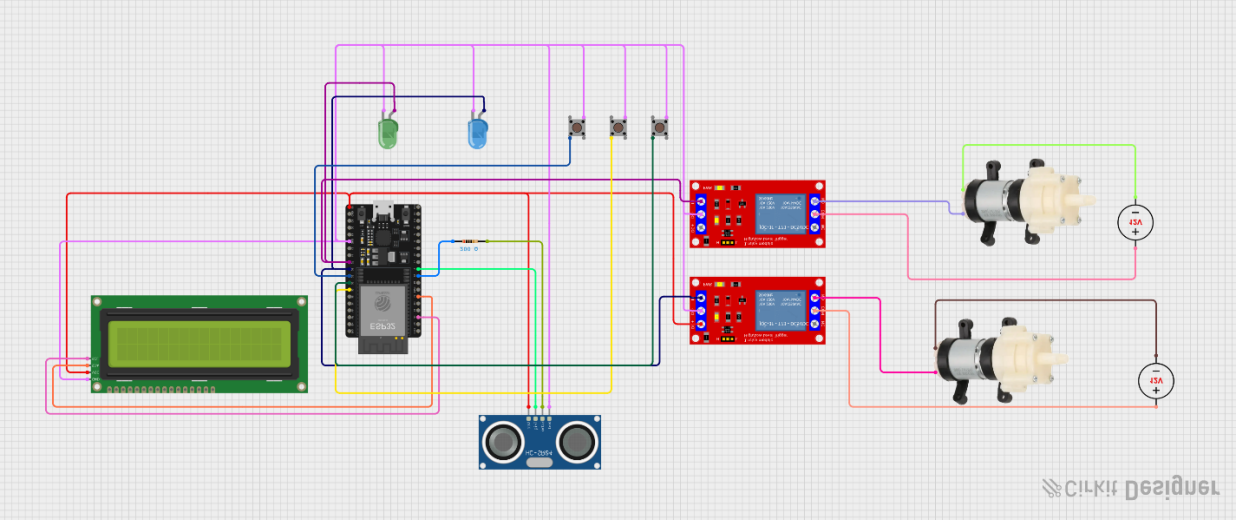
```
  }

  // ----------------------------------------
  // Monitor serial
  // ----------------------------------------

  Serial.print("Distancia: ");
  if (tiempo == 0) {
    Serial.println("Fuera de rango");
  } else {
    Serial.print(distancia, 1);
    Serial.print(" cm   |   Nivel: ");
    Serial.print((int)porcentaje);
    Serial.println("%");
  }

  delay(300);
}
```

Diagrama (REPOSITORIO)



Resultados

Tras la ejecución de las pruebas la práctica fue completamente satisfactoria pues todo salió exactamente según lo que pedía la práctica demostrando una precisión

estable en la lectura del sensor ultrasónico una interfaz de usuario fluida y sin interrupciones mediante los tres botones y un accionamiento automatizado impecable de las bombas simuladas en los umbrales programados mientras la pantalla mostraba de forma clara y sin errores la información en tiempo real validando la eficacia del código desarrollado directamente sobre el hardware del microcontrolador. En la parte física se pudo ver como con forme el programa estaba en ejecución las bombas iban llenando y baseando el contenedor cuando los códigos iban marcando el nivel de porcentaje especificado en la progracion que indicaban si se debía llenar o basear el agua, siendo exitoso todo el procedimiento.

Conclusión

En conclusión la elaboración de este sistema de control automatizado permitió consolidar de manera definitiva los conocimientos teóricos y prácticos sobre el manejo de microcontroladores modernos la interfaz con sensores y actuadores físicos y el diseño de lógicas de control embebido sin depender de ecosistemas de abstracción avanzada como Arduino lo cual fortaleció la comprensión profunda de la configuración de registros y drivers nativos cumpliendo a cabalidad con todos los requerimientos y objetivos planteados.

Referencias

Espressif Systems. (2025). *ESP32 Series Datasheet: Technical Specification*. Espressif Systems.

SparkFun Electronics. (2024). *HC-SR04 Ultrasonic Sensor Hookup Guide*. SparkFun Reference documentation.

Ogata, K. (2010). *Ingeniería de Control Moderna* (5ª ed.). Pearson Educación.